LeetCode 235 — Lowest Common Ancestor of a Binary Search Tree

1. Problem Title & Link

Title: LeetCode 235: Lowest Common Ancestor of a Binary Search Tree
Link: https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/

2. Problem Statement (Short Summary)

You are given a **Binary Search Tree (BST)** and two nodes p and q.

Your task:

👉 Find the **Lowest Common Ancestor (LCA)** of p and q.

📌 **Lowest Common Ancestor**:

- The **lowest node** in the tree that has **both p and q as descendants**
- A node can be a descendant of itself

3. Examples (Input → Output)

Example 1

Input:

root = [6,2,8,0,4,7,9,null,null,3,5]

p = 2, q = 8

Output:

6

Example 2

Input:

root = [6,2,8,0,4,7,9,null,null,3,5]

p = 2, q = 4

Output:

2

(One node is ancestor of the other)

Example 3

Input:

root = [2,1]

p = 2, q = 1

Output:

2

4. Constraints

- Number of nodes: $2 \le n \le 10^5$
- Node values are **unique**
- Tree is a **valid BST**
- Both p and q exist in the tree

5. Core Concept (Pattern / Topic)

Binary Search Tree Property + Greedy Traversal

We **do not** need full tree traversal.

6. Thought Process (Step-by-Step Explanation)

❌ **Wrong Thinking**

- Treating BST as normal binary tree
- Using generic LCA (LC 236) logic ❌ (overkill)

✔ **Correct Thinking — Use BST Ordering**

At any node root:

- If p.val and q.val are **both less** than root.val
  → LCA lies in **left subtree**
- If p.val and q.val are **both greater** than root.val
  → LCA lies in **right subtree**
- Otherwise
  → current root is the **LCA**

This works because of **BST ordering**.

7. Visual / Intuition Diagram (ASCII)

BST:

```
          6
        /  \
       2    8
      / \  / \
     0  4 7  9
       / \
      3   5
```

Case: p = 2, q = 8

2 < 6 < 8 → split happens here

LCA = 6

8. Pseudocode (Language Independent)

```
current = root

while current is not null:
    if p.val < current.val AND q.val < current.val:
        current = current.left
    else if p.val > current.val AND q.val > current.val:
        current = current.right
    else:
        return current
```

9. Code Implementation

✅ Python (Iterative — BEST)

```
class Solution:
    def lowestCommonAncestor(self, root, p, q):
        while root:
            if p.val < root.val and q.val < root.val:
                root = root.left
            elif p.val > root.val and q.val > root.val:
                root = root.right
            else:
                return root
```

✅ Python (Recursive)

```
class Solution:
    def lowestCommonAncestor(self, root, p, q):
        if not root:
            return None

        if p.val < root.val and q.val < root.val:
            return self.lowestCommonAncestor(root.left, p, q)
        elif p.val > root.val and q.val > root.val:
            return self.lowestCommonAncestor(root.right, p, q)
        else:
            return root
```

✅ Java (Iterative — Interview Preferred)

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        while (root != null) {
            if (p.val < root.val && q.val < root.val) {
                root = root.left;
            } else if (p.val > root.val && q.val > root.val) {
                root = root.right;
            } else {
                return root;
            }
        }
        return null;
    }
}
```

10. Time & Space Complexity

Time

- **O(h)** — height of BST
  - Balanced: O(log n)
  - Skewed: O(n)

Space

- Iterative: **O(1)**
- Recursive: **O(h)**

11. Common Mistakes / Edge Cases

❌ **Mistakes**

- Using normal binary tree LCA logic
- Not considering p or q can be ancestor
- Traversing both subtrees unnecessarily

✔ **Edge Cases**

- p is ancestor of q
- q is ancestor of p
- Root itself is LCA

12. Detailed Dry Run (Step-by-Step Table)

Input:

p = 2, q = 4

Node	Comparison	Action
6	both < 6	go left
2	split (2 ≤ 2 ≤ 4)	return 2

Answer = **2**

13. Common Use Cases (Real-Life / Interview)

- Hierarchical data (org charts)
- File system directory trees
- Database index trees
- Dependency resolution

14. Common Traps (Important!)

- Forgetting BST condition
- Assuming LCA must be strictly between p and q
- Overcomplicating solution

15. Builds To (Related LeetCode Problems)

- LC 236 — LCA of a Binary Tree
- LC 701 — Insert into BST
- LC 450 — Delete Node in BST
- LC 98 — Validate BST

16. Alternate Approaches + Comparison

✔️ 1️⃣ **BST Property (BEST)**

- Simple
- Optimal

2️⃣ **Parent Pointer Map**

- Extra space
- Unnecessary

17. Why This Solution Works (Short Intuition)

BST ordering guarantees that the first node where p and q diverge (or one equals the current node) is their **lowest common ancestor**.

18. Variations / Follow-Up Questions

- LCA in a normal binary tree (LC 236)

- LCA with parent pointers
- LCA of multiple nodes
- LCA queries with preprocessing